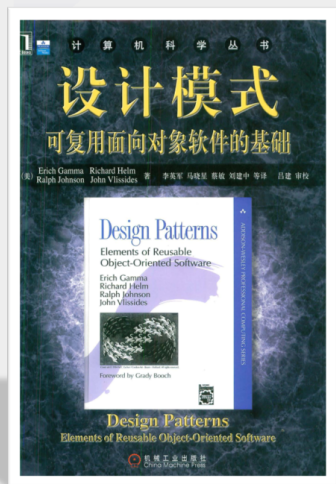




设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

4.1 Adapter（适配器）

一、意图（Intent）

将一个类的接口转换成客户希望的另外一个接口。

Adapter 使得原本由于接口不兼容而不能一起工作的类可以一起工作。

在嵌入式领域，这通常表现为：

- 上层业务依赖某固定 API（如 FreeRTOS API）
- 但底层环境（如 PC 上的 POSIX）接口不同
- 通过适配器，将 POSIX 接口转换为 FreeRTOS 接口

二、又称（Also Known As）

Wrapper（包装器）

三、动机（Motivation）

假设：

- 应用代码基于 FreeRTOS：

```
xTaskCreate(task, ...);
```

- 现在希望在 PC 上运行该代码
- PC 环境只有 `pthread_create`

问题：

- `pthread_create` 接口形式不同

- 业务代码已经大量依赖 FreeRTOS API
- 不希望修改应用层

解决方案：

- 写一个适配层
- 在 PC 环境下，用 `pthread_create` 实现 `xTaskCreate`

于是：



这样无需修改应用层即可在 PC 运行。

四、适用性 (Applicability)

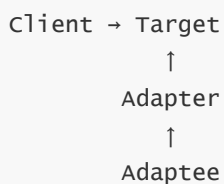
以下情况适合使用 Adapter：

1. 需要使用一个已存在的类，而其接口不符合你的需求
(例如 pthread 与 FreeRTOS API 不一致)
2. 希望创建一个可复用的类，与不相关或不可预见接口协同工作
3. 在嵌入式系统中：
 - RTOS 切换
 - HAL 统一
 - 老代码迁移
 - PC 仿真运行嵌入式逻辑

五、结构 (Structure)

在 C 语言中通常采用“对象适配器”方式（通过组合实现）。

结构关系：



在本例中：

- Client：应用任务代码
- Target：FreeRTOS API (`xTaskCreate`)
- Adaptee：`pthread_create`
- Adapter：`freertos_compat_posix.c`

六、参与者 (Participants)

1. Target

定义客户使用的特定领域接口。

在嵌入式场景中：

FreeRTOS 的任务 API

2. Client

与 Target 接口协作的对象。

例如：

业务任务代码

3. Adaptee

已经存在的接口，需要被适配。

例如：

POSIX pthread 接口

4. Adapter

将 Adaptee 的接口转换为 Target 接口。

在本例中：

- `xTaskCreate()` 内部调用 `pthread_create()`
 - 完成参数转换和封装
-

七、协作 (Collaborations)

Client 通过 Target 接口调用 Adapter。

Adapter：

1. 接收 Target 请求
2. 转换为 Adaptee 请求
3. 调用 Adaptee
4. 返回结果给 Client

在嵌入式示例中：

- 应用调用 `xTaskCreate`
- Adapter 将其转为 `pthread_create`
- pthread 创建线程
- 返回结果

八、效果 (Consequences)

优点

1. 将接口转换与业务逻辑分离
2. 提高可复用性
3. 减少对既有代码的修改
4. 支持平台迁移和环境仿真

缺点

1. 增加一层间接性
2. 可能无法完全匹配语义差异
(例如调度优先级、栈管理等)

在嵌入式中常见问题：

- FreeRTOS 的实时调度语义无法完全由 pthread 等价实现
- 适配器只能解决“接口不兼容”，不能保证“行为完全一致”

九、实现 (Implementation)

在 C 语言中没有继承机制，因此通常采用：

- 函数封装
- 结构体 + 函数指针
- 模块级封装

适配器实现关键点：

1. 不修改 Client
2. 不修改 Adaptee
3. 只在 Adapter 内完成接口转换

十、示例代码 (简要)

目标接口 (Target)：

```
BaseType_t xTaskCreate(TaskFunction_t task, ...);
```

适配器内部实现：

```
BaseType_t xTaskCreate(...)
{
    return pthread_create(...);
}
```

Client 无需修改。

十一、相关模式 (Related Patterns)

- **Bridge**
Bridge 旨在分离抽象与实现，使两者可以独立变化。
Adapter 用于事后接口兼容。
 - **Decorator**
Decorator 在不改变接口的情况下增加职责；
Adapter 改变接口。
 - **Facade**
Facade 提供统一接口，但不一定解决接口不兼容问题。
-

嵌入式场景总结

在嵌入式领域，Adapter 常见于：

- RTOS API 兼容层
- HAL 驱动统一
- 第三方库封装
- PC 仿真环境

核心思想不变：

当接口已经确定且不可修改时，通过一层适配器，使不兼容接口协同工作。